

《软件体系结构与设计模式》

作业报告

作业名称: 作业 1 数据处理软件系统架构分析与实践

授课教师: 毛斐巧

报告人: _____ 学号: _____ 班级: _____

报告提交时间: 2026年4月23日

成 績:

1.作业内容与要求:

(说明:鼓励大家有效利用 AI 工具完成作业,务必指出使用哪些 AI 工具,具体给了工具哪些提示指令来达成你的目的。)

在真实工业系统中,大量软件并不是 Web 或 App,而是数据处理系统。例如:生产数据分析系统、金融交易数据处理系统、供应链数据系统、AI 训练数据清洗系统。这些系统有三个特点:

□1) 数据正确性高于一切

□2) 需求会不断变化

□3) 系统必须长期维护

因此:一个优秀的软件工程师不仅要写出能运行的代码,更要设计出可扩展、可维护、可复用的软件架构。本次作业通过一个真实工业数据处理任务,学习掌握面向对象设计思想、软件架构设计、数据建模、工程级代码质量、自动化测试和可扩展系统设计。

下面是来自真实工业生产项目需求的数据预处理任务,给定一组按月为单位的 HT_工序记录明细(见附件 Excel,每月一份),要求实现下述功能:

1) 要求为每个批号、物料品号收集以下 8 个工艺参数向量:缆芯外径、护套外径、挤出内模、挤出外模、螺杆速度、螺杆电流、生产速度和实际生产速度。要求收集到的结果不遗漏、不错误,即每个批号原本有 8 个工艺参数,则不能遗漏,收集到 8 个参数值与原始数值一致。不遗漏和不错误的要求,需要开发和给出相应的测试用例加以证明。注意:数据处理型的软件不同于常见的 Web/App 应用开发,数据正确性是第一要求。

2) 构建 MySQL 数据库表来存放 1) 中的数据,以实现统计每个物料品号(一个物料品号有多个批号)的工艺参数向量的种类、工艺参数向量的统计分布情况。这里尚有许多统计需求未列出(如统计每个工艺参数的分布区间、聚类分析),后续实践内容可能会加上,因此,需要你在编写本项目代码时,保持对未来需求的预见能力,使得本次作业的软件代码和架构具有可扩展性、可维护性、可复用性、可修改性,以应对未来实践作业带来的新需求。

2.解答报告正文

理解和分析报告、实践过程及结果等。

1.数据理解

针对 HT_工序纪录的多个月的 Excel,试回答下述问题:

8 个工艺参数的单位是什么?

缆芯外径、护套外径、挤出内模、挤出外模: 毫米 (mm)

螺杆速度: 转/分 (rpm)

螺杆电流: 安培 (A)

生产速度、实际生产速度: 米/分 (m/min)

如何处理缺失值?

简单处理方法:丢弃或填充为 0/平均值。

强校验字段(如批号、缆芯外径等核心 8 参数):若缺失,该行数据视为脏数据,不可存入业务主表,应转移至 error_log 表,并记录缺失原因(如:“缺失螺杆电流”),触发数据异常

报警。

弱校验字段（如非关键温度）：可置为 NULL，并在后续统计时忽略。

如何处理某些批号具有非连续的字段信息？

时间序列对齐：使用“创建日期/时间”或“初始行号”作为时间戳进行排序。

窗口聚合：如果同一批号的数据被分割在不同文件或不同班次（如代码中提到的每 30 行为一个班次），需要在数据库层通过批号进行 GROUP BY 聚合操作。系统设计时需引入 BatchID 作为唯一追踪键，跨文件合并同批次数据。

字段之间的数学关系、取值范围、合法性检查的规则是什么？

物理常识约束：所有外径、速度、电流必须 >0 。

工艺包含约束：护套外径 (mm) $\geq$ 缆芯外径 (mm)；挤出外模 (mm) $\geq$ 挤出内模 (mm)。

衍生计算校验：diff-输入 必须等于 护套外径 - 缆芯外径；diff-挤出 必须等于 挤出外模 - 挤出内模（代码中存在此逻辑，需在 DB 或实体层设为自动计算属性或强校验规则）。

极值约束：需结合业务设定阈值（例如螺杆电流不能超过设备额定最大电流，实际生产速度应在理论生产速度的 $\pm 20\%$ 内）。

如何识别有问题的记录？

类型识别：本应是数值的字段出现了字符串（如“/”或“N/A”）。

越界识别：违反上述步骤 4 中定义的数学约束。

逻辑识别：如同一个批号在同一时间出现了两个差异极大的螺杆速度。

对这些数据理解，将形成如何设计数据库表的设计方案，要求：画出 ER 图，陈述：有多少个实体、实体之间是什么关系、字段约束是什么？

实体 (Entities)：

Material (物料)： 存储物料基础信息。

Batch (批次)： 记录生产批号信息（关联物料）。

ProcessRecord (工艺记录)： 记录具体的工艺参数（关联批次）。

ER 图关系陈述：

Material (1) -> (N) Batch： 一个物料品号包含多个生产批号。

Batch (1) -> (N) ProcessRecord： 一个批号包含多条时序工艺记录。

字段约束设计方案：

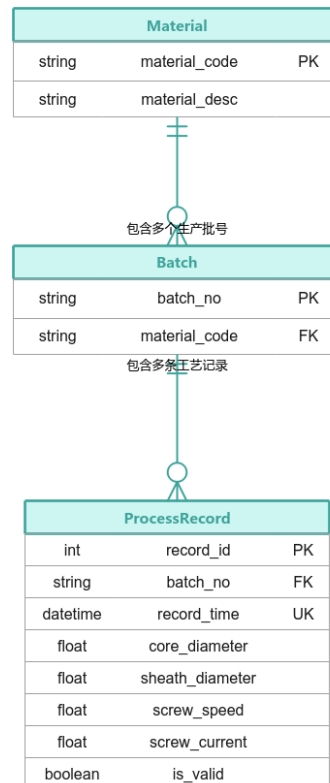
tb_material: material_code (PK, VARCHAR, NOT NULL), material_desc (VARCHAR).

tb_batch: batch_no (PK, VARCHAR, NOT NULL), material_code (FK, NOT NULL).

tb_process_record: record_id (PK, BIGINT auto_increment), batch_no (FK, NOT NULL),

record_time (DATETIME), core_diameter (DECIMAL(10,2), CHECK > 0), sheath_diameter (DECIMAL(10,2), CHECK > core_diameter), screw_speed (DECIMAL), screw_current (DECIMAL), is_valid (TINYINT).

生产工艺管理数据库ER图



2. 数据处理软件架构设计

依据上面的实验内容，试问答：

有哪些功能需要实现？

数据源适配（读取 Excel/CSV）

数据反序列化（将表格行映射为对象）

数据清洗与验证（校验缺失值、业务规则）

数据持久化（写入 MySQL）

统计与分析（按物料品号聚类、分布统计）

这些功能是如何在实体之间分配的？

ExcelDataLoader: 负责按块或流式加载 Excel 数据，隔离底层解析库（如 openpyxl）。

ProcessRecordEntity: 领域模型，封装一行数据的属性及自身的内部校验逻辑（如属性非空）。

ValidationEngine: 规则引擎，包含多个 IRule 实现（如 DiameterRule、CurrentRule）。

BatchRepository: 负责与数据库进行交互 (DAO 层), 处理 SQL 语句和连接。

StatisticsService: 负责从 Repository 获取数据并进行各类统计计算。

加装功能片段的实体称为对象, 对象之间是如何协作以实现功能的?

ExcelDataLoader (对象 A) 解析文件, 实例化并返回一组 **ProcessRecordEntity** (数据实体对象)。

协调者将这些实体对象传递给 **ValidationEngine** (规则引擎对象 B)。

引擎内部迭代调用事先注册的各个 **IRule** (具体规则对象 C, 如缺失值规则、外径差值规则) 进行合法性检查。

验证通过的实体对象被传递给 **BatchRepository** (持久化对象 D) 进行数据库入库, 或者交给 **StatisticsService** (统计服务对象 E) 进行指标聚类。

如何避免一个对象承担过多的工作从而避免单点维护和扩展故障?

严格遵循单一职责原则: 强行剥离“数据的获取”、“业务规则校验”、“数据的持久化”。例如, 绝不让读取 Excel 的类去承担外径相减的业务计算逻辑。

运用设计模式: 采用策略模式将各种校验规则和统计算法独立成单独的类; 采用门面模式对外提供统一的调用入口, 隐藏内部复杂的对象装配过程。

如何实现去中心化的对象协作模式?

管道-过滤器模式/ 责任链模式: 将多个数据处理节点串联, 数据如流水般经过过滤器, 没有任何一个“上帝控制类”去微操每一个步骤, 节点处理完只需调用 `next.process()`。

事件驱动 / 发布-订阅机制: 引入事件总线。当 **ExcelDataLoader** 读完一行数据时, 发布一个 **RecordParsedEvent**, 由独立的校验监听器和统计监听器自行订阅处理, 各个组件彻底解耦。

这组对象大约能支撑多大程度的需求变数?

增加新的工艺参数 (只需在实体类中新增属性及修改对应的数据表)。

增加新的清洗或校验规则 (只需新建实现 **IRule** 接口的类并注册到引擎, 主流程零修改)。

变更数据源或数据库底座 (由 Excel 改 CSV, 或 MySQL 改 PGSQL, 只需新增对应的具体实现类)。

什么样的合理需求将导致这组对象在加装更多的功能片段情况下也无法胜任?

数据吞吐量发生质变: 如果需求从“每月处理几个 Excel 离线跑批”变成“直接接入车间物联网(IIoT)传感器, 每秒推送上万条实时数据”, 这种基于单机内存聚合、同步阻塞式的面向对象架构将触发内存溢出, 必须重构为基于 **Kafka + Flink/Spark** 的分布式流处理架构。

强在线交互分析需求：如果客户要求在前端页面实时拖拽生成这 8 个参数的多维交叉对比图表，当前的纯数据预处理架构缺乏 MVC Web 服务层，无法胜任。

3.代码质量统计

试编写软件统计：

本实验中的类有多少个？

0 个。该代码“面向过程”编程，没有任何类的定义。

每个类的方法数量？

不存在类，系统由全局命名空间下的 6 个独立函数组成（`get_project_root`, `getAllRecords`, `createRecordExcel`, `isNull`, `writeRecordToExcelAndRemoveNull`, `writeRecordToExcel`）。

每个方法中的有效代码行数？

`get_project_root`: 2 行

`isNull`: 6 行

`createRecordExcel`: 15 行

`getAllRecords`: 25 行

`writeRecordToExcelAndRemoveNull`: 30 行

`writeRecordToExcel`: 35 行

整个代码系统的按方法、按类来统计的代码行数分布？

100% 的代码集中在全局函数中。整体代码约 150 行，其中数据解析与文件读写（I/O 操作）占据了约 50%，业务逻辑（数据清洗、差值计算）占据约 40%，控制流逻辑约 10%。

过度集中的代码在哪些类或方法中？

代码过度集中在 `writeRecordToExcelAndRemoveNull` 和 `writeRecordToExcel` 两个函数中。这两个函数同时包揽了：文件路径截取与拼接、创建 Excel 表、遍历数据字典、业务逻辑计算（如 `diff` 的数学减法）、过滤脏数据、向 Excel 追加数据、保存文件等 7 项完全不相干的职责。

不同的功能是在一个类中实现还是跨越多个类实现？

全部功能挤在一个 Python 文件模块内实现，没有跨类拆分，缺乏模块化设计。

是否存在中心化的功能处理逻辑？

代码底部的 `if __name__ == '__main__':` 充当了中心化的驱动器，将特定的文件列表写死。

而 `writeRecord...` 函数作为一个巨大的“黑盒”，中心化地控制了数据从计算到落盘的所有细节。

类之间的调用接口是怎样的？

无接口定义。各函数之间靠直接的参数传递，且传递的核心数据结构是弱类型的嵌套字典。内部数据细节完全向外裸露，毫无封装性可言。

配置文件如何传递？

没有配置文件。所有的核心配置均被硬编码在全局变量或函数内部。例如提取字段表 `need_fields` 是全局变量；Excel 列索引偏移量 `col = 23`、每班次行数 `30`、文件路径等直接写死在逻辑代码中。

4. 代码中使用了哪些面向对象设计原则

采用这些设计原则是如何达到：类之间的松耦合、类内部的高内聚，这些设计原则如何应对本数据处理任务的未来变化？

①朴素的“数据抽象”与“对象化”思维（字典充当 DTO/POJO）

在缺乏正式实体类的情况下，代码没有将“批号”、“外径”、“速度”等参数作为零散的独立变量在函数间传来传去，而是创建了 `oneRecord = {}`（字典）来包裹这一行数据。

②利用函数边界实现初级的“封装”与“功能隐藏”

代码没有将所有逻辑平铺直叙地写在一个巨大的脚本流中，而是划分成了 `get_project_root()`, `getAllRecords()`, `createRecordExcel()`, `isNull()` 等独立函数。

③部分实现了“单一职责原则”

代码中存在职责非常单一、清晰的函数。例如 `isNull(record)` 函数。

④具备“配置隔离”与“开闭原则”的雏形

代码顶部定义了全局变量 `need_fields = ['批号', '缆芯外径(mm)', ...]`。

⑤追求代码复用 (DRY - Don't Repeat Yourself)

在最底部的执行入口处，作者定义了 `files = [...]` 列表，然后使用 `for file in files:` 循环遍历执行解析和写入逻辑。

5. 代码挑刺

附件已经有一份 python 代码部分实现了功能 1)，但是有很大的缺点，包括命名不合理、不是采用面向对象思想编写、没有考虑整体数据处理任务的软件体系结构，无法适应后续客户未说但仍然存在的项目需求，因此，不具有可扩展性、可维护性、可复用性、可修改性。要求实验者指出附件代码存在的问题，结合具体的代码段，说明不满足《阿里巴巴 Java 开发手册》一书的哪些程序设计基本要求、说明不满足哪些面向对象设计原则。

①泛滥的未经预先定义的常量，极差的可维护性

不满足的阿里规约：【强制】不允许任何未经预先定义的常量直接出现在代码中。（手册：一、编程规约 - (二)常量定义 - 1)

```
28     col = 23 # 只需要读取第 23 列(项目名称) 和第 24 列(项目记录结果)的值
29
30     recordTotalNum = int((sheet.max_row - 1) / 30)
```

面向对象与架构缺陷：将“每 30 行一个班次”和“第 23 列”写死，严重违反了开闭原则。假设下个月车间更换了 Excel 模板，在前面插入了一列，或单班次数据变更为 40 条，这段核心代码将直接瘫痪。应将其提取为配置文件中的配置项，或通过解析表头动态获取列索引。

②严重违反单一职责

不满足的阿里规约：【推荐】单个方法的总行数不超过 80 行，代码逻辑分清红花和绿叶（核心业务与共性方法提取）。（手册：一、编程规约 - (三)代码格式 - 11)

在 writeRecordToExcelAndRemoveNull 函数中，同时揉捏了以下逻辑：

字符串查找截取组装路径 (file_path.find('/'))

创建 Excel 表 (createRecordExcel)

业务逻辑计算：if filed == 'diff-输入(d)': row_data.append(float(record['护套外径(mm)']) - float(record['缆芯外径(mm)']))

脏数据过滤 (isNull)

Excel I/O 写回 (wb.save)

```
87 # 将所有记录写入对应的记录表中，并去除字段为空的记录
88 def writeRecordToExcelAndRemoveNull(file_path, allRecords):
89     for key in allRecords.keys():
90         pos = file_path.find('/')
91         fileName = file_path[:pos + 1] + key + '_' + file_path[pos + 1:]
92         createRecordExcel(fileName)
93
94     # 将 allRecords 中的记录放入一个二维列表中
95     data = []
96     for record in allRecords[key]:
97         row_data = []
98         if isNull(record):
99             continue
100
101         for filed in need_fields:
102             if filed == 'diff-输入(d)':
103                 row_data.append(float(record['护套外径(mm)']) - float(record['缆芯外径(mm)']))
104             elif filed == 'diff-挤出(d)':
105                 row_data.append(float(record['挤出外径(mm)']) - float(record['挤出内模(mm)']))
106             else:
107                 row_data.append(record[filed])
108         data.append(row_data)
109
110     # 打开 Excel 表
111     wb = openpyxl.load_workbook(fileName)
112     # 选择要操作的工作表
113     sheet = wb.active
114     # 将二维列表中的数据追加到 Excel 表中
115     for row_data in data:
116         sheet.append(row_data)
117     # 保存修改后的 Excel 文件
118     wb.save(fileName)
119
```

面向对象与架构缺陷：严重违反单一职责原则 (SRP)。业务规则 (diff 如何计算) 被死死绑定在了文件 IO 遍历操作里。一旦后续需求改为“将数据写入 MySQL” (如作业要求 2)，

这个包含了业务计算的方法必须全部推翻重写，代码毫无复用性。

③面向字典编程，缺乏领域模型

不满足的阿里规约：**【参考】**分层领域模型规约：DO/DTO/BO 的应用。（手册：六、工程结构 - 3）及**【强制】**POJO 类属性必须使用包装数据类型。（手册：一、编程规约 - (四)OOP 规约 - 11）

```
24 # allRecords 用于存放所有的记录，key 是 HT-xx，value 是对应的列表记录
25 # 每一条记录都有类似于 ['批号', '缆芯外径(mm)', '护套外径(mm)', 'diff-输入(d)...] 的字段
26 allRecords = {}
27 oneRecord = {} # oneRecord 用于存放一条记录
```

面向对象与架构缺陷：这导致封装性彻底缺失。字典是弱类型的“黑盒”，下游处理模块根本无法静态推断 record 中是否存在‘缆芯外径(mm)’这个键，极易容易引发 KeyError。正确的做法是定义具体的领域实体类（如 class ProcessRecordEntity），将 8 个工艺参数作为类的强类型属性，并在构造器中完成基础格式校验。

④缺失异常保护体系，程序极其脆弱

不满足的阿里规约：**【强制】**捕获异常是为了处理它，不要捕获了却什么都不处理而抛弃之。（手册：二、异常日志 - (一)异常处理 - 4）

```
103 row_data.append(float(record['护套外径(mm)']) - float(record['缆芯外径(mm)']))
104 elif filed == 'diff-挤出(d)':
105     row_data.append(float(record['挤出外径(mm)']) - float(record['挤出内模(mm)']))
106 else:
107     row_data.append(record[filed])
```

面向对象与架构缺陷：缺失健壮性与容错性设计。在真实工业现场，工人常会误输字母、空格或 N/A。上述代码未加任何 try-except 块，直接调用 float() 强制转换。一旦遇到非数字格式，会立即抛出 ValueError。在缺乏脏数据隔离容器的现状下，一条脏数据就能让整批几千条数据处理程序直接崩溃闪退。

⑤命名风格混杂，不符合工程规范

不满足的阿里规约：**【强制】**方法名、参数名统一使用 lowerCamelCase 风格，必须遵从驼峰形式；**【强制】**杜绝完全不规范的缩写，避免望文不知义。（手册：一、编程规约 - (一)命名风格 - 4 & 11）

```
16
17 # 读取给定工序记录明细查询表，得到对应的所有记录
18 def getAllRecords(file_path):
19     # 加载 Excel 文件
20     projDir = get_project_root()
21     wb = openpyxl.load_workbook(projDir + '/' + file_path)
22     sheet = wb.active
```

面向对象与架构缺陷：代码中同时存在下划线命名法（need_fields, row_data）和驼峰命名法（getAllRecords, writeRecordToExcel）。这显得极其随意，不符合工业级团队协同开发的工程规范。